Build a 3-layer stacking classifier and measure the performance

```
import numpy as np
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import pandas as pd
```

Step 1: Load Dataset-(WINE DATASET)

```
data = load_wine()

df=pd.DataFrame(data.data,columns=data.feature_names)
df['target'] = data.target
```

```
print(data.feature_names)
```

```
['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash', 'magnesium', 'total_phenols', 'flavanoids', 'nonflavanoid_phenols', 'p
```

```
df.head(4)
```

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins
0	14.23	1.71	2.43	15.6	127.0	2.80	3.06	0.28	2.29
1	13.20	1.78	2.14	11.2	100.0	2.65	2.76	0.26	1.28
2	13.16	2.36	2.67	18.6	101.0	2.80	3.24	0.30	2.81
3	14.37	1.95	2.50	16.8	113.0	3.85	3.49	0.24	2.18

Next steps: Generate code with df New interactive sheet

Step 2 : Split the Dataset

```
X = df.drop('target', axis=1)
y = df['target']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Step 3: Data Preprocessing

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
```

Step 4: Layer 1 (Base Models)

```
from sklearn.ensemble import StackingClassifier
```

```
level0 = [
    ("lr", LogisticRegression(max_iter=5000, solver='liblinear')),
    ("dt", DecisionTreeClassifier()),
    ("rf", RandomForestClassifier())

]
```

```
from sklearn.metrics import accuracy_score


lr = level0[0][1]
dt = level0[1][1]
rf=level0[2][1]

lr.fit(X_train, y_train)
dt.fit(X_train, y_train)
rf.fit(X_train, y_train)

pred_lr = lr.predict(X_test)
pred_dt = dt.predict(X_test)
pred_rf = rf.predict(X_test)


print("LR Accuracy:", accuracy_score(y_test, pred_lr))
print("DT Accuracy:", accuracy_score(y_test, pred_dt))
print("RF Accuracy:", accuracy_score(y_test, pred_rf))
```

```
LR Accuracy: 0.9722222222222222
DT Accuracy: 0.9444444444444444
RF Accuracy: 1.0
```

Step 5: Layer 2 (Intermediate Models)

```
level1 = StackingClassifier(
    estimators=level0,
    final_estimator=GradientBoostingClassifier(),
    cv=5,n_jobs=-1
)
```

```
level1.fit(X_train, y_train)
pred_l1 = level1.predict(X_test)

print("Level 1 Accuracy (GB on base models):", accuracy_score(y_test, pred_l1))
```

```
Level 1 Accuracy (GB on base models): 0.9722222222222222
```

Step 6: Layer 3 (Meta Model)

```
level2 = StackingClassifier(
    estimators=[("stack1", level1)],
    final_estimator=SVC(),
    cv=5
)
```

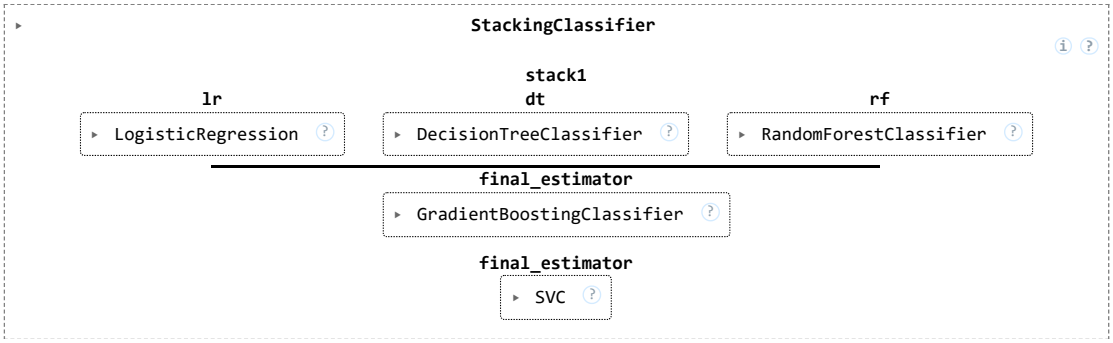
Step 7: Testing

```
level2.fit(X_train, y_train)
pred_l2 = level2.predict(X_test)

print("Level 2 Accuracy (Final SVC):", accuracy_score(y_test, pred_l2))
```

```
Level 2 Accuracy (Final SVC): 1.0
```

```
level2.fit(X_train, y_train)
```



Step 8: Evaluate Performance

```
y_pred = level2.predict(X_test)
```

```
print("3-Level Stacking Accuracy:", accuracy_score(y_test, y_pred))
```

```
3-Level Stacking Accuracy: 0.9722222222222222
```

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        14
           1       0.93      1.00      0.97        14
           2       1.00      0.88      0.93         8

    accuracy                           0.97        36
   macro avg       0.98      0.96      0.97        36
weighted avg       0.97      0.97      0.97        36
```